Web-Programmierung · Endpräsentation

CineRate

Filme suchen, bewerten und auf die Watchlist setzen

Nico Weber · Lukas Krebelder · Madlen Hahn · 09.11.2026

Projekt und Aufgabenverteilung

DAS PROJEKT

Punkt	Inhalt
Ziel	Webanwendung zum Suchen, Bewerten und Verwalten von Filmen
Funktionen	Konto, Filmsuche mit Genre-Filter, Detailseite, Bewertung (1–5 Sterne), Watchlist, Kommentare, Filmvorschläge
Aufbau	SPA und MPA, ein BFF (REST und GraphQL), Backend, Postgres
Filmdaten	aus der TMDB-API
Vorgehen	Scrum mit 4 Sprints

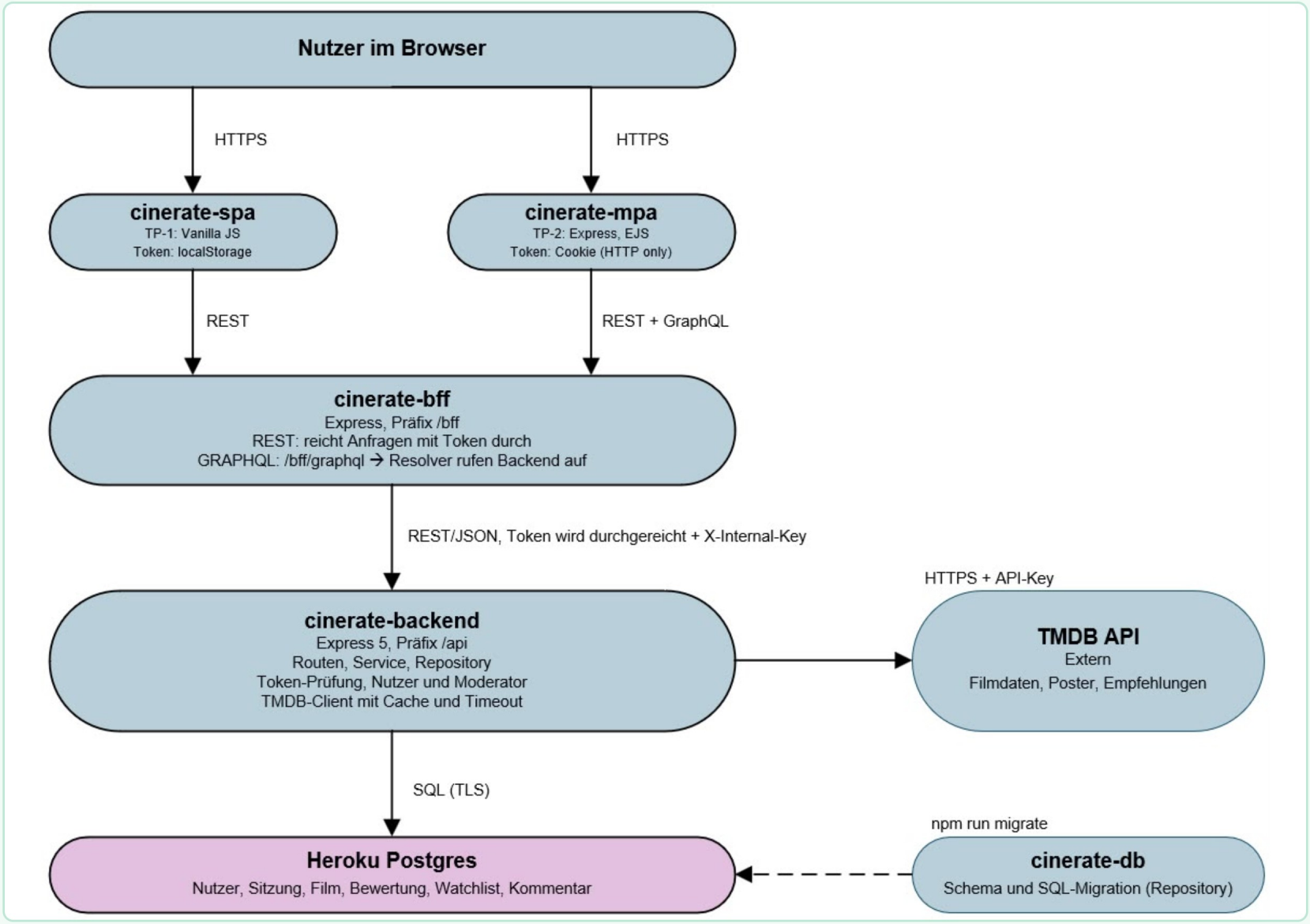
TEAM

Nico Weber
Product Owner, Developer: SPA, MPA

Lukas Krebelder
Scrum Master, Developer: Backend, Datenbank

Madlen Hahn
Developer: BFF

Architekturskizze



API-Design: Aufbau

Kern-Backend

/api

Hier liegen alle Routen und die Logik

←

BFF, REST

/bff

Dieselben Routen, Antwort und Status unverändert

+

BFF, GraphQL

/bff/graphql

Alle 17 Routen auch per GraphQL, Resolver rufen das Backend auf

So ist jede Route dokumentiert: Methode, Ressource, Data In, Data Out

Method	Ressource	Data In	Data Out

POST	/auth	{
		"email" : "lena.hartmann@example.com",
		"password" : "Kino2026!"
		}
			200
			{
			"token" : "a7f3k92xq1m8d5v0",
			"userId" : "u_58213"
			}
			400
			{
			"message" : "Email and password are required."
			}
			401
			{
			"message" : "Email or password is incorrect."
			}

DELETE	/auth	Header
		Authorization : "Bearer a7f3k92xq1m8d5v0"
			200
			{
			"message" : "Logout successful."
			}

API-Design: Ressourcen und Regeln

Ressource	Methoden	Zweck
/status	GET	Läuft das System?
/users	POST, DELETE	Registrieren, Konto löschen
/auth	POST, DELETE	Anmelden, Abmelden
/movies	GET	Suchen, filtern, Details
/movies/{id}/ratings	POST, GET	Bewerten
/movies/{id}/comments	GET, POST, PUT, DELETE	Kommentieren
/watchlist	GET, POST, DELETE	Watchlist
/recommendations	GET	Filmvorschläge

Token

Header „Authorization: Bearer …“, außer bei Status, Anmelden, Registrieren

Fehler

Immer gleiches Format: { "message": … }

Statuscodes

400 ungültig · 401 kein Token · 403 fremde Daten · 404 nicht gefunden · 409 doppelt · 429 zu viele Fehlversuche · 500 unerwarteter Fehler · 503 TMDB oder DB weg

Backend: drei Schichten

Beispiel: Lena bewertet einen Film mit 4 Sternen

Route

routes/ratings.js

```
router.post(
  "/movies/:movieId/ratings",
  requireAuth,
  async (req, res) ⇒ {
    const { created, ...body }
      = await ratingService
        .saveRating({ ... });
    res.status(
      created ? 201 : 200
    ).json(body);
  });
```

→ Service

services/ratingService.js

```
if (!Number.isInteger(value)
    || value < 1
    || value > 5) {
  throw new AppError(400, …);
}
const film = await movieService
  .ensureMovie(movieId);
await ratingRepository
  .saveRating({ ... });
```

→ Repository

db/ratingRepository.js

```
INSERT INTO Bewertung
  (nutzer_id, film_id,
   punktzahl)
SELECT $1, f.id, $3
  FROM Film f
  WHERE f.tmdb_id = $2
ON CONFLICT
  (nutzer_id, film_id)
DO UPDATE SET
  punktzahl =
    EXCLUDED.punktzahl,
  aktualisiert_am = NOW()
```

199 Tests laufen ohne Datenbank, weil der Service sein Repository übergeben bekommt.

Backend: drei Entscheidungen

1 **Zufälliges Token statt JWT**

`services/authService.js`

```
const { token, tokenHash } = createToken();
await sessionRepository.createSession(user.id, tokenHash);
return { token, userId: "u_" + user.id, rolle: user.rolle };
```

2 **Cache vor TMDB**

`integrations/cachedTmdbClient.js`

```
searchMovies: ({ query, page = 1 }) ⇒ {
  const key = `search:${String(query).trim().toLowerCase()}:${page}`;
  return cache.getOrLoad(key,
    () ⇒ tmdbClient.searchMovies({ query, page }), 10 * MINUTE);
},
```

3 **Nur das BFF darf ans Backend**

`middleware/requireInternalKey.js`

```
const given = req.get("X-Internal-Key") || "";
if (!crypto.timingSafeEqual(sha256(given), expectedHash)) {
  return next(new AppError(403, "Access is only allowed via the BFF."));
}
```

Product Backlog: Muss, Soll, Kann

29 Muss

Ohne das erfüllt CineRate seinen Zweck nicht

29 von 29 umgesetzt

- Konto: registrieren, anmelden, abmelden, löschen
- Filme: TMDB, Suche, Poster, Genre-Filter, Details
- Bewerten und Watchlist
- Personalisierte Filmvorschläge
- MPA mit Login und Filmsuche
- Technik: Schichten, GraphQL, Cache, Tests, CI
- Doku und Präsentation

6 Soll

Macht CineRate deutlich besser

5 von 6 umgesetzt

- Kommentare schreiben, bearbeiten, löschen
- Rollen Nutzer und Moderator
- Filter nach FSK-Freigabe **(offen)**
- Filmdetailseite in der MPA

6 Kann

Schön, wenn Zeit bleibt

Bewusst nicht umgesetzt

- Schauspieler auf der Detailseite
- Link zum Trailer
- FSK-Filter in der MPA
- Hintergrundauftrag für Filmdaten
- Nutzer suchen und hinzufügen
- Eindeutiger Nutzername

Live-Demo

- Registrieren und Anmelden
- Filme suchen und filtern
- Bewerten und kommentieren
- Watchlist
- Filmvorschläge „Für dich“

Danach: Ihre Fragen

ER-Diagramm (Anhang)

